
Virtualisation and Cloud Technologies



CEM YILMAZ
Faculty № 22572118
Technical University of Gabrovo
Faculty of
Electrical Engineering and Electronics

Project made for the degree of
Software and Computer Engineering
June 4, 2026

Contents

1	Introduction	2
2	Methodology	3
2.1	Test Environment	3
2.2	Hypervisor Configuration	4
2.3	Virtual Machine Configuration	4
2.4	Benchmark Tools	5
2.5	Benchmark Procedures	5
2.5.1	CPU Benchmark	5
2.5.2	Memory Benchmark	5
2.5.3	Disk I/O Benchmark	6
2.6	Controlled Variables	6
3	Results	7
3.1	Visualisation Methodology	7
3.2	CPU Benchmarks	8
3.3	Memory Benchmarks	8
3.4	Disk I/O Benchmarks	9
4	Discussion	9
5	Conclusion	9
	Appendices	11
A	CPU Benchmark Results	11
B	Memory Benchmark Results	12
C	Disk I/O Benchmark Results	13

1 Introduction

Virtualisation is a method of isolating and abstracting computing resources, allowing multiple virtual instances to run on a single physical machine. A virtual machine is defined as an efficient, isolated duplicate of a real machine (Popek & Goldberg, 1974). Popek and Goldberg categorise three essential characteristics of a virtual machine: an “essentially identical” environment, efficiency, and resource control (Popek & Goldberg, 1974).

Virtualisation has become a fundamental technology in modern computing, enabling the efficient use of hardware resources, improving scalability, and facilitating the deployment of applications in cloud environments. However, at the heart of virtualisation and its technologies lies two distinct methods of achieving it: the type-1 (bare-metal) and type-2 (hosted) hypervisors (Susnjara & Smalley, 2024).

Type-1 hypervisors run directly on the host’s physical hardware allowing it to directly interact with the underlying infrastructure such as the central processing unit (CPU), memory, graphical compute, and storage. The direct access to hardware resources allows type-1 hypervisors to provide better performance and scalability compared to type-2 hypervisors. Type-1 hypervisors are commonly used in enterprise environments and data centers where performance and security are critical. Examples of type-1 hypervisors include VMware ESXi, Microsoft Hyper-V, Citrix Hypervisor, Kernel-based Virtual Machine (KVM) and its derivatives e.g. Proxmox VE (Susnjara & Smalley, 2024).

Type-2 hypervisors, however, do not directly run in the underlying infrastructure. Instead, type-2 hypervisors run as a layer on top of the host’s existing operating system (OS). For communication with the physical layer, the type-2 hypervisor creates a bridge between the OS and the virtualised environment. This additional layer of abstraction can lead to performance overhead compared to type-1 hypervisors, as it directly affects the Popek and Goldberg criterion of efficiency. Specifically, a statistically dominant subset of instructions must execute directly on the physical processor without VMM intervention (Popek & Goldberg, 1974). Type-2 hypervisors are often used for desktop virtualisation and testing environments where ease of use and flexibility are more important than raw performance. Examples of type-2 hypervisors include VMWare workstation, Oracle VirtualBox, Microsoft Virtual PC (Susnjara & Smalley, 2024).

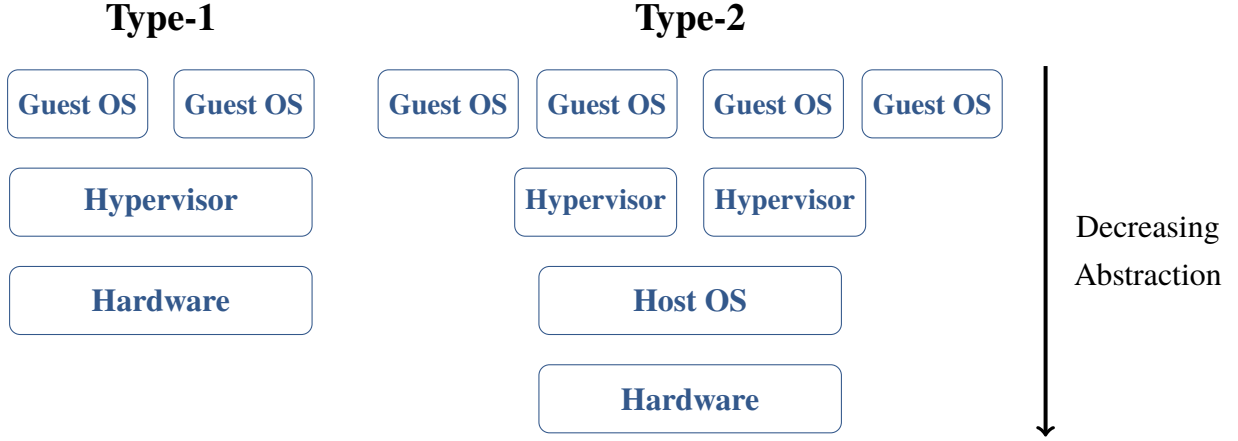


Figure 1: Architectural comparison of type-1 and type-2 hypervisors.

Modern virtualisation technologies have evolved to minimise overhead and improve performance for both types of hypervisors. For instance, hardware-assisted virtualisation features such as Intel VT-x and AMD-V have been introduced to enhance the performance of virtual machines by allowing certain instructions to be executed directly on the physical CPU, reducing the need for VMM intervention (Susnjara & Smalley, 2024). Additionally, advancements in memory management and I/O virtualisation have further improved the efficiency of virtualised environments (Barham et al., 2003). However, the question of whether measurable difference remain and under what workload conditions is the central concern of this paper.

This paper presents a comparative performance evaluation of type-1 and type-2 hypervisor technologies under controlled laboratory benchmarks. The methodology, results and analysis that follow aim to determine whether the theoretical performance advantage of type-1 hypervisors is still observable in practice today.

2 Methodology

2.1 Test Environment

All benchmarks were conducted on a single physical host machine to ensure consistency across both hypervisor configurations. The host hardware specifications are detailed in Table 1.

Table 1: Host machine hardware specifications.

Component	Specification
CPU	Intel Core i5-13600K (14 cores, 20 threads)
RAM	64 GB DDR4 @ 3200 MHz
Storage	Seagate Barracuda 2TB HDD (ST2000DM008, SATA III, 7200 RPM)
OS (Type 2 host)	Microsoft Windows 11 25H2 (Build 26200.8524)

2.2 Hypervisor Configuration

Two hypervisors were selected to represent each architectural type:

- **Type-1: Proxmox VE 9.2-1.** Proxmox VE uses KVM as its virtualisation backend and is classified as a Type 1 hypervisor (Djordjevic et al., 2022). It was installed directly on the physical hardware with no general purpose host OS present. Accessed at https://enterprise.proxmox.com/iso/proxmox-ve_9.2-1.iso.
- **Type-2: Oracle VirtualBox 7.2.8.** VirtualBox was installed on the Windows 11 host OS. As a hosted hypervisor, VirtualBox relies on Windows for hardware access, introducing an additional software layer between the guest VM and physical hardware. This is the architectural source of overhead being measured in this experiment. Accessed at <https://download.virtualbox.org/virtualbox/7.2.8/VirtualBox-7.2.8-173730-Win.exe>.

2.3 Virtual Machine Configuration

To isolate hypervisor architecture as the sole independent variable, both virtual machines were configured identically across all parameters. The guest operating system was **Ubuntu Server 26.04 LTS** in both cases, installed from the same ISO image and accessed at <https://releases.ubuntu.com/26.04/ubuntu-26.04-live-server-amd64.iso>. The VM specifications are summarised in Table 2.

Table 2: Virtual machine configuration for both hypervisors.

Parameter	Value
vCPUs	4
RAM	8 GB
Disk size	50 GB (fixed/pre-allocated)
Disk controller	SATA
Firmware	BIOS
Guest OS	Ubuntu Server 26.04 LTS

The disk was allocated as a fixed size image on both hypervisors rather than dynamically allocated, to eliminate allocation time overhead from influencing I/O benchmark results. The same disk controller type (SATA) was used on both VMs to avoid introducing paravirtualised driver advantages. By default, Proxmox VE pre-configures with VirtIO SCSI, which would give it an unfair I/O advantage and confound the comparison.

Swap was disabled on both VMs prior to benchmarking (`sudo swapoff -a`) to ensure that memory benchmark results reflect RAM throughput exclusively, without spilling to disk.

2.4 Benchmark Tools

Two common benchmarking tools were used:

- **sysbench** used for CPU and memory benchmarks. For the purpose of this experiment, the version used was 1.0.20.
- **fiio** used for disk I/O benchmarks. For the purpose of this experiment, the version used was 3.41.

Both tools were installed from the Ubuntu package repositories on each VM independently.

2.5 Benchmark Procedures

Each benchmark was executed five times per VM, and the arithmetic mean of the five runs was taken as the reported result. Prior to each test session, the VM was booted and left idle for two minutes to allow background OS processes to settle. No other VMs or applications were running on the host during any test.

2.5.1 CPU Benchmark

The sysbench CPU benchmark was used to measure computational throughput. The test calculates prime numbers up to a configurable upper bound using all available vCPU threads, and reports the number of events completed per second.

```
1 sysbench cpu --cpu-max-prime=20000 --threads=4 --time=60 run
```

Listing 1: sysbench CPU benchmark command.

The `--threads=4` parameter matches the vCPU allocation of each VM, and the test duration was fixed at 60 seconds.

2.5.2 Memory Benchmark

The sysbench memory benchmark was used to measure memory subsystem throughput. The test performs sequential read and write operations using 1 MB blocks until a total transfer of 10 GB has been completed, reporting throughput in MiB/s.

```
1 sysbench memory --memory-block-size=1M --memory-total-size=10G --threads=4  
run
```

Listing 2: sysbench memory benchmark command.

2.5.3 Disk I/O Benchmark

Disk I/O performance was evaluated using fio with 4K random read and random write workloads, as small random I/O is the pattern most sensitive to hypervisor overhead. The I/O engine used was libaio (Linux asynchronous I/O), with 4 parallel jobs matching the vCPU count.

A throwaway fio run was performed before each workload type to prepare the disk cache, ensuring that subsequent measurements reflect steady I/O performance rather than cold cache behaviour. The warmup run uses the same name as the benchmark it precedes so that fio reuses the pre-created files during the actual benchmark runs. After the random read warmup and five benchmark runs, the randread files were deleted before performing the random write warmup.

```
1  fio --name=randread --ioengine=libaio --rw=randread \  
2      --bs=4k --size=64M --numjobs=4 --runtime=60 \  
3      --direct=1 --group_reporting
```

Listing 3: fio random read benchmark command.

```
1  fio --name=randwrite --ioengine=libaio --rw=randwrite \  
2      --bs=4k --size=64M --numjobs=4 --runtime=60 \  
3      --direct=1 --group_reporting
```

Listing 4: fio random write benchmark command.

2.6 Controlled Variables

Table 3 summarises the variables held constant across both test environments to ensure the validity of the comparison.

Table 3: Controlled variables across both hypervisor configurations.

Variable	Control measure
Guest OS	Identical Ubuntu Server 26.04 LTS install
vCPU count	4 on both VMs
RAM allocation	8 GB on both VMs
Disk size	50 GB fixed allocation on both VMs
Disk controller	SATA on both VMs
Firmware type	BIOS on both VMs
Swap	Disabled on both VMs
Concurrent load	No other VMs or applications running
Idle period	2 minutes post-boot before each test session
Benchmark commands	Identical commands on both VMs
Run count	5 runs per benchmark; arithmetic mean reported

3 Results

3.1 Visualisation Methodology

Each benchmark subsection presents results using a radar chart, which allows multiple heterogeneous performance metrics to be compared simultaneously on a single figure. Because the metrics differ in unit and magnitude, all values are normalised to the dimensionless range $[0, 1]$ before plotting.

Let m_i denote the arithmetic mean of metric i across the five benchmark runs, and let $R_i > 0$ be a predefined reference maximum for that metric. The normalised score $\hat{s}_i$ is given by:

$$\hat{s}_i = \frac{m_i}{R_i}, \text{ where } \hat{s}_i \in [0, 1] \quad (1)$$

The reference maximum R_i for each metric is chosen as a conservative upper bound that strictly exceeds all observed values across both hypervisor configurations, i.e.:

$$R_i = \kappa \cdot \max(m_i^{\text{VBox}}, m_i^{\text{Proxmox}}), \text{ where } \kappa > 1 \quad (2)$$

where κ is a headroom factor chosen such that no data point saturates its axis, ensuring the full scale of the chart remains interpretable. The identical reference maxima are applied to both hypervisors, guaranteeing that the two polygons are directly comparable.

For metrics where a lower value indicates better performance (e.g. latency, standard deviation), the score is inverted so that a larger enclosed polygon area consistently represents superior performance across all axes:

$$\hat{s}_i^{\text{inv}} = 1 - \hat{s}_i = 1 - \frac{m_i}{R_i} \quad (3)$$

Equations (1)–(3) are applied uniformly across the CPU, memory, and disk I/O benchmark sections that follow.

3.2 CPU Benchmarks

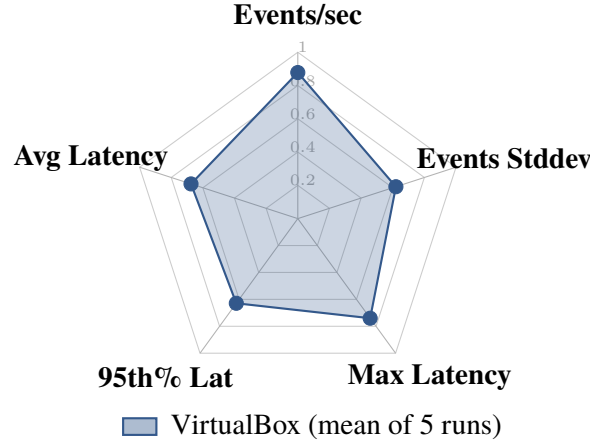


Figure 2: VirtualBox CPU benchmark performance radar chart.

Figure 2 shows the normalised performance profile of VirtualBox across five CPU metrics. Scores are computed using Equations (1)–(3), with the following reference maxima: Events/sec $R = 7,000$; Average Latency $R = 2$ ms; 95th Percentile Latency $R = 2$ ms; Maximum Latency $R = 20$ ms; Standard Deviation $R = 500$. Latency and standard deviation axes are inverted per Equation (3), such that a larger enclosed area indicates better overall performance.

3.3 Memory Benchmarks

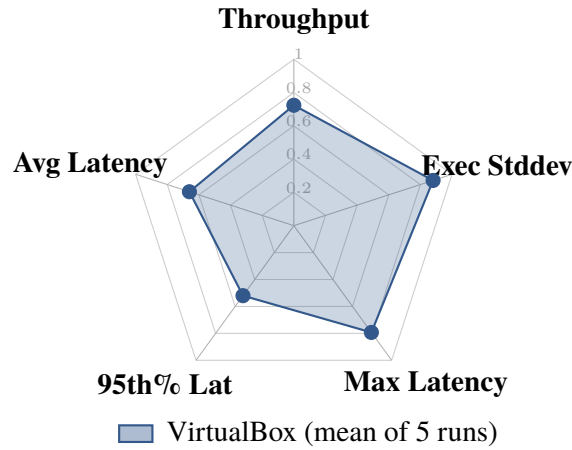


Figure 3: VirtualBox memory benchmark performance radar chart.

Figure 3 shows the normalised memory performance profile of VirtualBox across five metrics. Scores are computed using Equations (1)–(3), with the following reference maxima: Throughput $R = 120,000$ MiB/s; Average Latency $R = 0.10$ ms; 95th Percentile Latency $R = 0.10$ ms; Maximum Latency $R = 15$ ms; Execution Standard Deviation $R = 0.05$. Latency and standard deviation axes are inverted per Equation (3).

3.4 Disk I/O Benchmarks

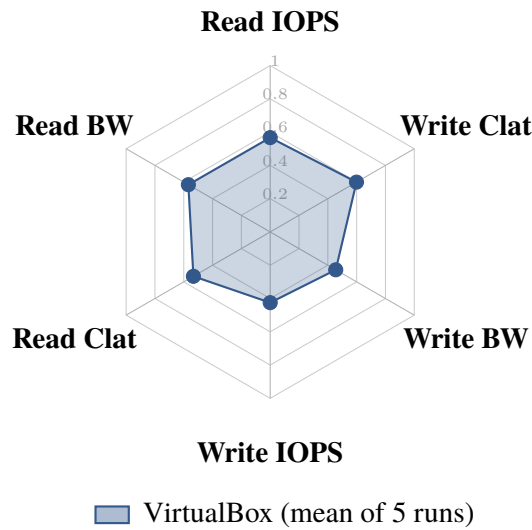


Figure 4: VirtualBox disk I/O benchmark performance radar chart.

Figure 4 shows the normalised disk I/O performance profile of VirtualBox across six metrics covering both random read and random write workloads. Scores are computed using Equations (1)–(3), with the following reference maxima: Read IOPS $R = 500$; Read Bandwidth $R = 2,000$ KiB/s; Read Completion Latency $R = 30,000$ μ s; Write IOPS $R = 400$; Write Bandwidth $R = 1,500$ KiB/s; Write Completion Latency $R = 60,000$ μ s. Completion latency axes are inverted per Equation (3).

4 Discussion

5 Conclusion

References

- Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I., & Warfield, A. (2003). Xen and the art of virtualization. *37*, 164–177. <https://doi.org/10.1145/1165389.945462>
- Djordjevic, B., Timcenko, V., Kraljevic, N., & Jovicic, N. (2022, November 15). *Performance comparison of KVM and Proxmox Type-1 Hypervisors*. <https://doi.org/10.1109/TELFOR56187.2022.9983666>
- Popek, G. J., & Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, *17*(7), 412–421. <https://doi.org/10.1145/361011.361073>
- Susnjara, S., & Smalley, I. (2024, October 30). *What Are Hypervisors?* | IBM. Retrieved June 3, 2026, from <https://www.ibm.com/think/topics/hypervisors>

Appendices

A CPU Benchmark Results

Table 4: VirtualBox CPU benchmark results (sysbench, 5 runs).

Run	Events/sec	Total Events	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Events Stddev
1	6138.21	368301	0.62	0.65	7.31	0.74	101.23
2	6177.91	370689	0.62	0.65	2.48	0.73	159.81
3	6135.72	368153	0.62	0.65	11.05	0.75	375.66
4	6161.86	369721	0.62	0.65	2.46	0.74	124.00
5	6102.55	366163	0.62	0.65	2.69	0.75	189.52
Mean	6143.25	368605	0.62	0.65	5.20	0.742	190.04

Table 5: Proxmox CPU benchmark results (sysbench, 5 runs).

Run	Events/sec	Total Events	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Events Stddev
1							
2							
3							
4							
5							
Mean							

B Memory Benchmark Results

Table 6: VirtualBox memory benchmark results (sysbench, 5 runs).

Run	Throughput (MiB/s)	Total Ops	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Exec Stddev
1	106235.51	10240	0.02	0.03	3.17	0.04	0.00
2	67435.16	10240	0.02	0.03	0.22	0.05	0.01
3	69355.60	10240	0.02	0.03	0.19	0.04	0.01
4	103618.39	10240	0.02	0.04	1.47	0.07	0.00
5	87759.48	10240	0.02	0.04	10.45	0.04	0.01
Mean	86880.83	10240	0.02	0.034	3.10	0.048	0.006

Table 7: Proxmox memory benchmark results (sysbench, 5 runs).

Run	Throughput (MiB/s)	Total Ops	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Exec Stddev
1							
2							
3							
4							
5							
Mean							

C Disk I/O Benchmark Results

Table 8: VirtualBox random read disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μ s)	Clat Stdev (μ s)
1	291	1167	1165.61	290.20	51.85	12.97	13608.85	14213.95
2	278	1113	1114.32	277.19	52.09	13.01	14274.81	15848.32
3	269	1076	1075.96	267.15	52.86	13.24	14771.76	16303.08
4	282	1130	1129.46	280.81	48.03	12.04	14048.93	14851.57
5	297	1190	1191.12	296.57	55.46	13.87	13337.12	15130.93
Mean	283.4	1135.2	1135.29	282.38	52.06	13.03	14008.29	15269.57

Table 9: Proxmox random read disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μ s)	Clat Stdev (μ s)
1								
2								
3								
4								
5								
Mean								

Table 10: VirtualBox random write disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μ s)	Clat Stdev (μ s)
1	133	534	3930.80	980.98	996.44	249.30	29862.54	402831.82
2	142	568	3032.12	756.62	917.81	229.52	28033.35	445593.33
3	178	713	5303.47	1324.18	1150.74	287.79	22326.45	383498.96
4	189	758	6071.94	1517.17	1305.09	326.34	20985.67	353770.39
5	209	836	5295.56	1322.22	1262.74	315.85	19019.98	316261.22
Mean	170.2	681.8	4726.78	1180.23	1126.56	281.76	24045.60	380391.14

Table 11: Proxmox random write disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μ s)	Clat Stdev (μ s)
1								
2								
3								
4								
5								
Mean								